Studentai 2026 VT

Generated by Doxygen 1.17.0

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

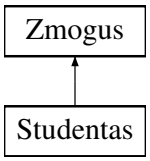
Chapter 4

Class Documentation

4.1 Studentas Class Reference

`#include <studentai.h>`

Inheritance diagram for Studentas:



Public Member Functions

- Studentas (const string &v="A", const string &p="BB", const vector< int > &paz_={}, int egz_=0)
- ~Studentas ()
- Studentas (const Studentas &other)
- Studentas & operator= (const Studentas &other)
- Studentas (Studentas &&other) noexcept
- Studentas & operator= (Studentas &&other) noexcept
- void info () const override
- vector< int > & refPaz ()
- int & refEgz ()
- double & refRez ()
- double & refMedrez ()
- void clearPaz ()
- double getRez () const
- double getMedrez () const
- int getEgz () const
- vector< int > & getPaz ()
- void addPaz (int p)
- void setEgz (int e)
- void skaiciuoti ()

Public Member Functions inherited from Zmogus

- Zmogus (const string &v="A", const string &p="BB")
- virtual ~Zmogus ()
- string & refVardas ()
- string & refPavarde ()
- string getVardas () const
- string getPavarde () const
- void setVardas (const string &v)
- void setPavarde (const string &p)

Additional Inherited Members

Protected Attributes inherited from Zmogus

- string vardas
- string pavarde

4.1.1 Detailed Description

Definition at line 45 of file studentai.h.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Studentas() [1/3]

```
Studentas::Studentas (
            const string & v = "A",
            const string & p = "BB",
            const vector< int > & paz_ = {},
            int egz_ = 0)  [inline]
```

Definition at line 53 of file studentai.h.

4.1.2.2 ~Studentas()

```
Studentas::~Studentas ()  [inline]
```

Definition at line 70 of file studentai.h.

4.1.2.3 Studentas() [2/3]

```
Studentas::Studentas (
            const Studentas & other)  [inline]
```

Definition at line 77 of file studentai.h.

4.1.2.4 Studentas() [3/3]

```
Studentas::Studentas (
            Studentas && other)  [inline], [noexcept]
```

Definition at line 96 of file studentai.h.

4.1.3 Member Function Documentation

4.1.3.1 addPaz()

```
void Studentas::addPaz (
            int p)  [inline]
```

Definition at line 138 of file studentai.h.

4.1.3.2 clearPaz()

```
void Studentas::clearPaz ()  [inline]
```

Definition at line 131 of file studentai.h.

4.1.3.3 getEgz()

```
int Studentas::getEgz () const  [inline]
```

Definition at line 134 of file studentai.h.

4.1.3.4 getMedrez()

```
double Studentas::getMedrez () const  [inline]
```

Definition at line 133 of file studentai.h.

4.1.3.5 getPaz()

```
vector< int > & Studentas::getPaz ()  [inline]
```

Definition at line 136 of file studentai.h.

4.1.3.6 getRez()

```
double Studentas::getRez () const  [inline]
```

Definition at line 132 of file studentai.h.

4.1.3.7 info()

```
void Studentas::info () const  [inline], [override], [virtual]
```

Implements Zmogus.

Definition at line 123 of file studentai.h.

4.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (
            const Studentas & other)  [inline]
```

Definition at line 85 of file studentai.h.

4.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (
            Studentas && other)  [inline], [noexcept]
```

Definition at line 108 of file studentai.h.

4.1.3.10 refEgz()

```
int & Studentas::refEgz ()  [inline]
```

Definition at line 128 of file studentai.h.

4.1.3.11 refMedrez()

```
double & Studentas::refMedrez ()  [inline]
```

Definition at line 130 of file studentai.h.

4.1.3.12 refPaz()

```
vector< int > & Studentas::refPaz ()  [inline]
```

Definition at line 127 of file studentai.h.

4.1.3.13 refRez()

```
double & Studentas::refRez ()  [inline]
```

Definition at line 129 of file studentai.h.

4.1.3.14 setEgz()

```
void Studentas::setEgz (
            int e)  [inline]
```

Definition at line 139 of file studentai.h.

4.1.3.15 skaiciuoti()

```
void Studentas::skaiciuoti ()
```

Compute final average and median-based results

Definition at line 17 of file funk.cpp.

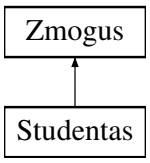
The documentation for this class was generated from the following files:

- studentai.h
- funk.cpp

4.2 Zmogus Class Reference

```
#include <studentai.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- Zmogus (const string &v="A", const string &p="BB")
- virtual ~Zmogus ()
- virtual void info () const =0
- string & refVardas ()
- string & refPavarde ()
- string getVardas () const
- string getPavarde () const
- void setVardas (const string &v)
- void setPavarde (const string &p)

Protected Attributes

- string vardas
- string pavarde

4.2.1 Detailed Description

Definition at line 23 of file studentai.h.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Zmogus()

```
Zmogus::Zmogus (
            const string & v = "A",
            const string & p = "BB")  [inline]
```

Definition at line 29 of file studentai.h.

4.2.2.2 ~Zmogus()

```
virtual Zmogus::~Zmogus ()  [inline], [virtual]
```

Definition at line 32 of file studentai.h.

4.2.3 Member Function Documentation

4.2.3.1 getPavarde()

```
string Zmogus::getPavarde () const  [inline]
```

Definition at line 39 of file studentai.h.

4.2.3.2 getVardas()

```
string Zmogus::getVardas () const  [inline]
```

Definition at line 38 of file studentai.h.

4.2.3.3 info()

```
virtual void Zmogus::info () const  [pure virtual]
```

Implemented in Studentas.

4.2.3.4 refPavarde()

```
string & Zmogus::refPavarde ()  [inline]
```

Definition at line 37 of file studentai.h.

4.2.3.5 refVardas()

```
string & Zmogus::refVardas ()  [inline]
```

Definition at line 36 of file studentai.h.

4.2.3.6 setPavarde()

```
void Zmogus::setPavarde (
            const string & p)  [inline]
```

Definition at line 41 of file studentai.h.

4.2.3.7 setVardas()

```
void Zmogus::setVardas (
            const string & v)  [inline]
```

Definition at line 40 of file studentai.h.

4.2.4 Member Data Documentation

4.2.4.1 pavarde

```
string Zmogus::pavarde  [protected]
```

Definition at line 26 of file studentai.h.

4.2.4.2 vardas

```
string Zmogus::vardas  [protected]
```

Definition at line 25 of file studentai.h.

The documentation for this class was generated from the following file:

- studentai.h

Chapter 5

File Documentation

5.1 funk.cpp File Reference

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <string>
#include <cmath>
#include <algorithm>
#include <random>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include "studentai.h"
#include <stdexcept>
```

Functions

- void inputas (vector< Studentas > &grupe)
- void outputas (const vector< Studentas > &grupe)
- void failoutputas (const vector< Studentas > &grupe, string writing)
- void randominputas (vector< Studentas > &grupe, bool ArGeneruotiVardus)
- void failinputas (vector< Studentas > &grupe, string reading)
- string randomvardas (string lytis)
- string randompavarde (string lytis)
- string lytgen ()
- bool cmpVardas (const Studentas &a, const Studentas &b)
- bool cmpPavarde (const Studentas &a, const Studentas &b)
- bool cmpRez (const Studentas &a, const Studentas &b)
- bool cmpMedrez (const Studentas &a, const Studentas &b)
- bool cmpEgz (const Studentas &a, const Studentas &b)
- void rikiuoti (vector< Studentas > &grupe, int pasirinkimas)
- void failgeneravimas ()
- void genirasymas (vector< Studentas > &grupe, vector< Studentas > &protingi, vector< Studentas > &neprotingi, string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija)

- void skirstyti (const vector< Studentas > &grupe, vector< Studentas > &protingi, vector< Studentas > &neprotingi)
- void logResults (const string &container, int size, double read_t, double sort_t, double split_t)
- void skirstyti2 (vector< Studentas > &grupe, vector< Studentas > &neprotingi)
- void skirstyti3 (vector< Studentas > &grupe, vector< Studentas > &neprotingi)
- ostream & operator<< (ostream &os, const Studentas &s)
- istream & operator>> (istream &is, Studentas &s)

5.1.1 Function Documentation

5.1.1.1 cmpEgz()

```
bool cmpEgz (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 363 of file funk.cpp.

5.1.1.2 cmpMedrez()

```
bool cmpMedrez (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 358 of file funk.cpp.

5.1.1.3 cmpPavarde()

```
bool cmpPavarde (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 348 of file funk.cpp.

5.1.1.4 cmpRez()

```
bool cmpRez (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 353 of file funk.cpp.

5.1.1.5 cmpVardas()

```
bool cmpVardas (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 343 of file funk.cpp.

5.1.1.6 failgeneravimas()

```
void failgeneravimas ()
```

Definition at line 384 of file funk.cpp.

5.1.1.7 failinputas()

```
void failinputas (
            vector< Studentas > & grupe,
            string reading)
```

Definition at line 259 of file funk.cpp.

5.1.1.8 failoutputas()

```
void failoutputas (
            const vector< Studentas > & grupe,
            string writing)
```

Write students to file

Definition at line 141 of file funk.cpp.

5.1.1.9 genirasymas()

```
void genirasymas (
            vector< Studentas > & grupe,
            vector< Studentas > & protingi,
            vector< Studentas > & neprotingi,
            string writing_good,
            string writing_bad,
            int rik_pasirinkimas,
            SkirstymoStrategija strategija)
```

Definition at line 418 of file funk.cpp.

5.1.1.10 inputas()

```
void inputas (
            vector< Studentas > & grupe)
```

Definition at line 38 of file funk.cpp.

5.1.1.11 logResults()

```
void logResults (
            const string & container,
            int size,
            double read_t,
            double sort_t,
            double split_t)
```

Definition at line 555 of file funk.cpp.

5.1.1.12 lytgen()

```
string lytgen ()
```

Definition at line 337 of file funk.cpp.

5.1.1.13 operator<<()

```
ostream & operator<< (
            ostream & os,
            const Studentas & s)
```

Definition at line 595 of file funk.cpp.

5.1.1.14 operator>>()

```
istream & operator>> (
            istream & is,
            Studentas & s)
```

Definition at line 604 of file funk.cpp.

5.1.1.15 outputas()

```
void outputas (
            const vector< Studentas > & grupe)
```

Print students to console

Definition at line 120 of file funk.cpp.

5.1.1.16 randominputas()

```
void randominputas (
            vector< Studentas > & grupe,
            bool ArGeneruotiVardus)
```

Definition at line 166 of file funk.cpp.

5.1.1.17 randompavarde()

```
string randompavarde (
            string lytis)
```

Definition at line 324 of file funk.cpp.

5.1.1.18 randomvardas()

```
string randomvardas (
            string lytis)
```

Definition at line 316 of file funk.cpp.

5.1.1.19 rikiuoti()

```
void rikiuoti (
            vector< Studentas > & grupe,
            int pasirinkimas)
```

Definition at line 369 of file funk.cpp.

5.1.1.20 skirstyti()

```
void skirstyti (
            const vector< Studentas > & grupe,
            vector< Studentas > & protingi,
            vector< Studentas > & neprotingi)
```

Definition at line 543 of file funk.cpp.

5.1.1.21 skirstyti2()

```
void skirstyti2 (
            vector< Studentas > & grupe,
            vector< Studentas > & neprotingi)
```

Definition at line 567 of file funk.cpp.

5.1.1.22 skirstyti3()

```
void skirstyti3 (
            vector< Studentas > & grupe,
            vector< Studentas > & neprotingi)
```

Definition at line 577 of file funk.cpp.

5.2 funk.cpp

Go to the documentation of this file.

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 void Studentas::skaiciuoti()
00018 {
00020     if (paz.empty()) return;
00021
00022     int sum = 0;
00023     for (int x : paz) sum += x;
00024
00025     rez = 0.4 * (sum * 1.0 / paz.size()) + 0.6 * egz;
00026
00027     vector<int> temp = paz;
00028     sort(temp.begin(), temp.end());
00029
00030     if (temp.size() % 2 == 0)
00031         medrez = (temp[temp.size()/2 - 1] + temp[temp.size()/2]) / 2.0;
00032     else
00033         medrez = temp[temp.size()/2];
00034
00035     medrez = medrez * 0.4 + egz * 0.6;
00036 }
00037
00038 void inputas(vector<Studentas> &grupe)
00039 {
00040     Studentas A;
00041
00042     while (true)
00043     {
00044         int sum = 0;
00045
00046         cout « "Vardas ir pavarde: ";
00047         cin » A.refVardas() » A.refPavarde();
00048
00049         int i = 0;
00050         string ats;
00051         while (true)
00052         {
00053             int temp;
00054             cout « "Iveskite pazymi numeris " « i + 1 « ": ";
00055             try
00056             {
00057                 cin » temp;
00058                 if (cin.fail())
00059                     throw runtime_error("Bloga ivestis.");
00060             }
00061             catch (exception& e)
00062             {
00063                 cin.clear();
00064                 cin.ignore(10000, '\n');
00065                 cout « e.what() « " Bandykite dar karta.\n";
00066                 continue;
00067             }
00068
00069             A.refPaz().push_back(temp);
00070             sum += temp;
00071
00072             cout « "Ar toliau vesite pazymius? (y/n): ";
00073             cin » ats;
00074             if (ats == "n") break;
00075             else if (ats == "y") cout « "Veskite toliau: ";
00076
00077             i++;
00078         }
00079
00080         cout « "Iveskite egzamino rezultata: ";
00081         try
00082         {
00083             cin » A.refEgz();
```

```
00084             if (cin.fail())
00085                 throw runtime_error("Bloga ivestis.");
00086         }
00087         catch (exception& e)
00088         {
00089             cin.clear();
00090             cin.ignore(10000, '\n');
00091             cout << e.what() << " Bandykite dar karta.\n";
00092             continue;
00093         }
00094 
00095         if (!A.refPaz().empty())
00096         {
00097             A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00098 
00099             sort(A.refPaz().begin(), A.refPaz().end());
00100 
00101             int n = A.refPaz().size();
00102             if (n % 2 == 0)
00103                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00104             else
00105                 A.refMedrez() = A.refPaz()[n/2];
00106 
00107             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00108             A.skaiciuoti();
00109         }
00110 
00111         grupe.push_back(A);
00112         A.clearPaz();
00113 
00114         cout << "Ar toliau vesite mokinius? (y/n): ";
00115         cin >> ats;
00116         if (ats == "n") break;
00117     }
00118 }
00119 
00120 void outputas(const vector<Studentas> &grupe)
00121 {
00123     cout << "-------------------------------------------------------------------------" << endl;
00124     cout << left << setw(20) << "Vardas"
00125          << setw(20) << "Pavarde"
00126          << setw(20) << "Rezultatas (Vid.) /"
00127          << setw(20) << " Rezultatas (Med.)"
00128          << endl;
00129     cout << "-------------------------------------------------------------------------" << endl;
00130 
00131     for (const auto &A : grupe)
00132     {
00133         cout << left << setw(20) << A.getVardas()
00134              << setw(20) << A.getPavarde()
00135              << right << setw(20) << fixed << setprecision(2) << A.getRez()
00136              << setw(20) << A.getMedrez()
00137              << endl;
00138     }
00139 }
00140 
00141 void failoutputas(const vector<Studentas> &grupe, string writing)
00142 {
00144     ofstream out(writing);
00145     out << "------------------------------------------------------------------------- \n";
00146     out << left << setw(20) << "Vardas"
00147         << setw(20) << "Pavarde"
00148         << setw(20) << "Rezultatas (Vid.) /"
00149         << setw(20) << " Rezultatas (Med.)"
00150         << "\n";
00151     out << "------------------------------------------------------------------------- \n";
00152 
00153     for (const Studentas& s : grupe)
00154     {
00155         out << left << setw(20) << s.getVardas()
00156             << setw(20) << s.getPavarde()
00157             << fixed << setprecision(2)
00158             << setw(20) << s.getRez()
00159             << setw(20) << s.getMedrez()
00160             << '\n';
00161     }
00162 
00163     cout << "Studentu skaicius: " << grupe.size() << endl;
00164 }
00165 
00166 void randominputas(vector<Studentas> &grupe, bool ArGeneruotiVardus)
00167 {
00168     Studentas A;
00169     string lytis;
00170     string ats;
00171 
00172     if (ArGeneruotiVardus)
```

```
00173     {
00174         int zmonsk = rand() % 100 + 1;
00175 
00176         for(int iii = 0; iii < zmonsk; iii++)
00177         {
00178             int sum = 0;
00179 
00180             lytis = lytgen();
00181             A.refVardas() = randomvardas(lytis);
00182             A.refPavarde() = randompavarde(lytis);
00183 
00184             int pazsk = rand() % 100 + 1;
00185 
00186             for (int ii = 0; ii < pazsk; ii++)
00187             {
00188                 int temp = rand() % 10 + 1;
00189                 A.refPaz().push_back(temp);
00190                 sum += temp;
00191             }
00192 
00193             A.refEgz() = rand() % 10 + 1;
00194 
00195             if (!A.refPaz().empty())
00196             {
00197                 A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00198 
00199                 sort(A.refPaz().begin(), A.refPaz().end());
00200 
00201                 int n = A.refPaz().size();
00202                 if (n % 2 == 0)
00203                     A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00204                 else
00205                     A.refMedrez() = A.refPaz()[n/2];
00206 
00207                 A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00208             }
00209 
00210             grupe.push_back(A);
00211             A.clearPaz();
00212         }
00213     }
00214     else
00215     {
00216         while (true)
00217         {
00218             int sum = 0;
00219 
00220             cout << "Vardas ir pavarde: ";
00221             cin >> A.refVardas() >> A.refPavarde();
00222 
00223             int pazsk = rand() % 100 + 1;
00224 
00225             for (int i = 0; i < pazsk; i++)
00226             {
00227                 int temp = rand() % 10 + 1;
00228                 A.refPaz().push_back(temp);
00229                 sum += temp;
00230             }
00231 
00232             A.refEgz() = rand() % 10 + 1;
00233 
00234             if (!A.refPaz().empty())
00235             {
00236                 A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00237 
00238                 sort(A.refPaz().begin(), A.refPaz().end());
00239 
00240                 int n = A.refPaz().size();
00241                 if (n % 2 == 0)
00242                     A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00243                 else
00244                     A.refMedrez() = A.refPaz()[n/2];
00245 
00246                 A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00247             }
00248 
00249             grupe.push_back(A);
00250             A.clearPaz();
00251 
00252             cout << "Ar toliau vesite zmones? (y/n): ";
00253             cin >> ats;
00254             if (ats == "n") break;
00255         }
00256     }
00257 }
00258 
00259 void failinputas(vector<Studentas> &grupe, string reading)
```

```
00260 {
00261     ifstream file(reading);
00262     grupe.reserve(1000000);
00263 
00264     if (!file)
00265     {
00266         cout « "Nepavyko atidaryti failo\n";
00267         return;
00268     }
00269 
00270     string line;
00271     getline(file, line);
00272 
00273     while (getline(file, line))
00274     {
00275         istringstream iss(line);
00276 
00277         Studentas A;
00278 
00279         iss » A.refVardas() » A.refPavarde();
00280 
00281         int pazymys;
00282 
00283         while (iss » pazymys)
00284             A.refPaz().push_back(pazymys);
00285 
00286         if (!A.refPaz().empty())
00287         {
00288             A.refEgz() = A.refPaz().back();
00289             A.refPaz().pop_back();
00290         }
00291 
00292         if (!A.refPaz().empty())
00293         {
00294             int sum = 0;
00295             for (int x : A.refPaz())
00296                 sum += x;
00297 
00298             sort(A.refPaz().begin(), A.refPaz().end());
00299 
00300             int n = A.refPaz().size();
00301 
00302             if (n % 2 == 0)
00303                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00304             else
00305                 A.refMedrez() = A.refPaz()[n/2];
00306 
00307             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00308             A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00309         }
00310 
00311         grupe.push_back(A);
00312     }
00313 }
00314 
00315 
00316 string randomvardas(string lytis)
00317 {
00318     if (lytis == "mot")
00319         return fvardai[rand() % fvardai.size()];
00320     else
00321         return mvardai[rand() % mvardai.size()];
00322 }
00323 
00324 string randompavarde(string lytis)
00325 {
00326     int index = rand() % pavardes.size();
00327     string galune;
00328 
00329     if (lytis == "mot")
00330         galune = fgalunes[rand() % fgalunes.size()];
00331     else
00332         galune = mgalunes[rand() % mgalunes.size()];
00333 
00334     return pavardes[index] + galune;
00335 }
00336 
00337 string lytgen()
00338 {
00339     return rand() % 2 == 0 ? "vyr" : "mot";
00340 }
00341 
00342 
00343 bool cmpVardas(const Studentas &a, const Studentas &b)
00344 {
00345     return a.getVardas() < b.getVardas();
00346 }
```

```
00347
00348 bool cmpPavarde(const Studentas &a, const Studentas &b)
00349 {
00350     return a.getPavarde() < b.getPavarde();
00351 }
00352
00353 bool cmpRez(const Studentas &a, const Studentas &b)
00354 {
00355     return a.getRez() < b.getRez();
00356 }
00357
00358 bool cmpMedrez(const Studentas &a, const Studentas &b)
00359 {
00360     return a.getMedrez() < b.getMedrez();
00361 }
00362
00363 bool cmpEgz(const Studentas &a, const Studentas &b)
00364 {
00365     return a.getEgz() < b.getEgz();
00366 }
00367
00368
00369 void rikiuoti(vector<Studentas> &grupe, int pasirinkimas)
00370 {
00371     if (pasirinkimas == 1)
00372         sort(grupe.begin(), grupe.end(), cmpVardas);
00373     else if (pasirinkimas == 2)
00374         sort(grupe.begin(), grupe.end(), cmpPavarde);
00375     else if (pasirinkimas == 3)
00376         sort(grupe.begin(), grupe.end(), cmpRez);
00377     else if (pasirinkimas == 4)
00378         sort(grupe.begin(), grupe.end(), cmpMedrez);
00379     else if (pasirinkimas == 5)
00380         sort(grupe.begin(), grupe.end(), cmpEgz);
00381 }
00382
00383
00384 void failgeneravimas()
00385 {
00386     int sk = 1000;
00387     for (int it = 0; it < 5; it++)
00388     {
00389         cout « "pradedamas " « sk « " eiluciu failo generavimo laiko skaiciavimas" « endl;
00390         auto input_start = chrono::high_resolution_clock::now();
00391
00392         ofstream gen("gen" + to_string(sk) + ".txt");
00393         gen « left « setw(20) « "STUD. VARDAS"
00394             « setw(20) « "STUD. PAVARDE"
00395             « setw(20) « "STUD PAZYMIAI (PASKUTINIS EGZAMINO)\n";
00396
00397         for (int genit = 1; genit <= sk; genit++)
00398         {
00399             gen « left « setw(20) « "genVardas" + to_string(genit)
00400                 « setw(20) « "genPavarde" + to_string(genit);
00401
00402             for (int i = 1; i <= 16; i++)
00403                 gen « setw(20) « rand() % 10 + 1;
00404
00405             gen « "\n";
00406         }
00407
00408         gen.close();
00409         sk *= 10;
00410
00411         auto input_end = chrono::high_resolution_clock::now();
00412         chrono::duration<double> input_diff = input_end - input_start;
00413         cout « "Failu sukurimas + uzdarymas uztruko: " « input_diff.count() « " s \n";
00414     }
00415 }
00416
00417
00418 void genirasymas(vector<Studentas> &grupe, vector<Studentas> &protingi, vector<Studentas> &neprotingi,
00419     string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija)
00420 {
00421     int sk = 1000;
00422
00423     for (int it = 0; it < 5; it++)
00424     {
00425         string filename = "gen" + to_string(sk) + ".txt";
00426
00427         cout « "Pradedamas failo nuskaitymas: " « filename « endl;
00428         auto start_read = chrono::high_resolution_clock::now();
00429
00430         ifstream file(filename);
00431
00432         if (!file)
00433         {
```

```
00434             cout « "Nepavyko atidaryti failo\n";
00435             sk *= 10;
00436             continue;
00437         }
00438 
00439         string line;
00440 
00441         getline(file, line); // skip header
00442 
00443         while (getline(file, line))
00444         {
00445             istringstream iss(line);
00446 
00447             Studentas A;
00448 
00449             iss » A.refVardas() » A.refPavarde();
00450 
00451             int paz;
00452             while (iss » paz)
00453                 A.refPaz().push_back(paz);
00454 
00455             if (A.refPaz().empty())
00456                 continue;
00457 
00458             A.refEgz() = A.refPaz().back();
00459             A.refPaz().pop_back();
00460 
00461             int sum = 0;
00462             for (int x : A.refPaz())
00463                 sum += x;
00464 
00465             A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00466 
00467             sort(A.refPaz().begin(), A.refPaz().end());
00468 
00469             int n = A.refPaz().size();
00470 
00471             if (n % 2 == 0)
00472                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00473             else
00474                 A.refMedrez() = A.refPaz()[n/2];
00475 
00476             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00477 
00478             grupe.push_back(A);
00479         }
00480 
00481         file.close();
00482 
00483         auto end_read = chrono::high_resolution_clock::now();
00484         chrono::duration<double> diff_read = end_read - start_read;
00485 
00486         cout « "Failo nuskaitymas uztruko: " « diff_read.count() « " s\n";
00487 
00488         auto start_rikiavimas = chrono::high_resolution_clock::now();
00489         rikiuoti(grupe, rik_pasirinkimas);
00490         auto end_rikiavimas = chrono::high_resolution_clock::now();
00491         chrono::duration<double> diff_rikiavimas = end_rikiavimas - start_rikiavimas;
00492 
00493         cout « "Rikiavimas failui " « filename « " uztruko: " « diff_rikiavimas.count() « " s\n";
00494 
00495         auto start_skirstymas = chrono::high_resolution_clock::now();
00496 
00497         if (strategija == SkirstymoStrategija::PIRMAS)
00498             skirstyti(grupe, protingi, neprotingi);
00499         else if (strategija == SkirstymoStrategija::ANTRAS)
00500             skirstyti2(grupe, neprotingi);
00501         else if (strategija == SkirstymoStrategija::TRECIAS)
00502             skirstyti3(grupe, neprotingi);
00503 
00504         auto end_skirstymas = chrono::high_resolution_clock::now();
00505         chrono::duration<double> diff_skirstymas = end_skirstymas - start_skirstymas;
00506 
00507         cout « "Protingu/Neprotingu skirstymas failui " « filename « " uztruko: " «
      diff_skirstymas.count() « " s\n";
00508 
00509         if (strategija == SkirstymoStrategija::PIRMAS)
00510         {
00511             cout « "Protingu ";
00512             failoutputas(protingi, "kursiokai_geri_" + to_string(sk) + ".txt");
00513             cout « "Neprotingu ";
00514             failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00515         }
00516         else
00517         {
00518             cout « "Protingu ";
00519             failoutputas(grupe, "kursiokai_geri_" + to_string(sk) + ".txt");
```

```
00520             cout « "Neprotingu ";
00521             failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00522         }
00523
00524         cout « "\n";
00525
00526         grupe.clear();
00527         protingi.clear();
00528         neprotingi.clear();
00529
00530         logResults(
00531             "vector",
00532             sk,
00533             diff_read.count(),
00534             diff_rikiavimas.count(),
00535             diff_skirstymas.count()
00536         );
00537
00538         sk *= 10;
00539     }
00540 }
00541
00542
00543 void skirstyti(const vector<Studentas>& grupe, vector<Studentas>& protingi, vector<Studentas>&
    neprotingi)
00544 {
00545     for (const auto& s : grupe)
00546     {
00547         if (s.getRez() >= 5.0)
00548             protingi.push_back(s);
00549         else
00550             neprotingi.push_back(s);
00551     }
00552 }
00553
00554
00555 void logResults(const string& container, int size,
00556                 double read_t, double sort_t, double split_t)
00557 {
00558     ofstream out("results.csv", ios::app);
00559     out « container « ","
00560         « size « ","
00561         « read_t « ","
00562         « sort_t « ","
00563         « split_t « "\n";
00564 }
00565
00566
00567 void skirstyti2(vector<Studentas>& grupe, vector<Studentas>& neprotingi)
00568 {
00569     auto it = partition(grupe.begin(), grupe.end(),
00570         [](const Studentas& s) { return s.getRez() >= 5.0; });
00571
00572     neprotingi.assign(it, grupe.end());
00573     grupe.erase(it, grupe.end());
00574 }
00575
00576
00577 void skirstyti3(vector<Studentas>& grupe, vector<Studentas>& neprotingi)
00578 {
00579     neprotingi.reserve(grupe.size());
00580
00581     auto it = remove_if(grupe.begin(), grupe.end(),
00582         [&](const Studentas& s)
00583         {
00584             if (s.getRez() < 5.0)
00585             {
00586                 neprotingi.push_back(s);
00587                 return true;
00588             }
00589             return false;
00590         });
00591
00592     grupe.erase(it, grupe.end());
00593 }
00594
00595 ostream& operator«(ostream& os, const Studentas& s)
00596 {
00597     os « left « setw(20) « s.getVardas()
00598        « setw(20) « s.getPavarde()
00599        « setw(10) « fixed « setprecision(2) « s.getRez()
00600        « setw(10) « s.getMedrez();
00601     return os;
00602 }
00603
00604 istream& operator»(istream& is, Studentas& s)
00605 {
```

```
00606     s.refPaz().clear();
00607
00608     is » s.refVardas() » s.refPavarde();
00609
00610     int paz;
00611     while (is » paz) {
00612         s.refPaz().push_back(paz);
00613     }
00614
00615     if (!s.refPaz().empty()) {
00616         s.refEgz() = s.refPaz().back();
00617         s.refPaz().pop_back();
00618         s.skaiciuoti();
00619     }
00620
00621     return is;
00622 }
```

5.3 studentai.h File Reference

```
#include <vector>
#include <string>
#include <algorithm>
```

Classes

- class Zmogus
- class Studentas

Enumerations

- enum class SkirstymoStrategija { PIRMAS , ANTRAS , TRECIAS }

Functions

- void inputas (vector< Studentas > &grupe)
- void failinputas (vector< Studentas > &grupe, string reading)
- void randominputas (vector< Studentas > &grupe, bool ArGeneruotiVardus)
- void outputas (const vector< Studentas > &grupe)
- void failoutputas (const vector< Studentas > &grupe, string writing)
- void failgeneravimas ()
- void genirasymas (vector< Studentas > &grupe, vector< Studentas > &protingi, vector< Studentas > &neprotingi, string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija)
- string lytgen ()
- string randomvardas (string lytis)
- string randompavarde (string lytis)
- void rikiuoti (vector< Studentas > &grupe, int pasirinkimas)
- bool cmpVardas (const Studentas &a, const Studentas &b)
- bool cmpPavarde (const Studentas &a, const Studentas &b)
- bool cmpRez (const Studentas &a, const Studentas &b)
- bool cmpMedrez (const Studentas &a, const Studentas &b)
- bool cmpEgz (const Studentas &a, const Studentas &b)
- void skirstyti (const vector< Studentas > &grupe, vector< Studentas > &protingi, vector< Studentas > &neprotingi)
- void logResults (const string &container, int size, double read_t, double sort_t, double split_t)
- void skirstyti2 (vector< Studentas > &grupe, vector< Studentas > &neprotingi)
- void skirstyti3 (vector< Studentas > &grupe, vector< Studentas > &neprotingi)
- ostream & operator<< (ostream &os, const Studentas &s)
- istream & operator>> (istream &is, Studentas &s)

Variables

- const vector< string > mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"}
- const vector< string > fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"}
- const vector< string > pavardes = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk", "Briedausk"}
- const vector< string > mgalunes = {"as", "aitis"}
- const vector< string > fgalunes = {"iene", "aite", "yte"}
- const string irasymo_failas = "kursiokai.txt"
- const string irasymo_failas_geras = "kursiokai_geri.txt"
- const string irasymo_failas_blogas = "kursiokai_blogi.txt"

5.3.1 Enumeration Type Documentation

5.3.1.1 SkirstymoStrategija

```
enum class SkirstymoStrategija  [strong]
```

Enumerator

PIRMAS	
ANTRAS	
TRECIAS	

Definition at line 16 of file studentai.h.

5.3.2 Function Documentation

5.3.2.1 cmpEgz()

```
bool cmpEgz (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 363 of file funk.cpp.

5.3.2.2 cmpMedrez()

```
bool cmpMedrez (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 358 of file funk.cpp.

5.3.2.3 cmpPavarde()

```
bool cmpPavarde (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 348 of file funk.cpp.

5.3.2.4 cmpRez()

```
bool cmpRez (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 353 of file funk.cpp.

5.3.2.5 cmpVardas()

```
bool cmpVardas (
            const Studentas & a,
            const Studentas & b)
```

Definition at line 343 of file funk.cpp.

5.3.2.6 failgeneravimas()

```
void failgeneravimas ()
```

Definition at line 384 of file funk.cpp.

5.3.2.7 failinputas()

```
void failinputas (
            vector< Studentas > & grupe,
            string reading)
```

Definition at line 259 of file funk.cpp.

5.3.2.8 failoutputas()

```
void failoutputas (
            const vector< Studentas > & grupe,
            string writing)
```

Write students to file

Definition at line 141 of file funk.cpp.

5.3.2.9 genirasymas()

```
void genirasymas (
            vector< Studentas > & grupe,
            vector< Studentas > & protingi,
            vector< Studentas > & neprotingi,
            string writing_good,
            string writing_bad,
            int rik_pasirinkimas,
            SkirstymoStrategija strategija)
```

Definition at line 418 of file funk.cpp.

5.3.2.10 inputas()

```
void inputas (
            vector< Studentas > & grupe)
```

Definition at line 38 of file funk.cpp.

5.3.2.11 logResults()

```
void logResults (
            const string & container,
            int size,
            double read_t,
            double sort_t,
            double split_t)
```

Definition at line 555 of file funk.cpp.

5.3.2.12 lytgen()

```
string lytgen ()
```

Definition at line 337 of file funk.cpp.

5.3.2.13 operator<<()

```
ostream & operator<< (
            ostream & os,
            const Studentas & s)
```

Definition at line 595 of file funk.cpp.

5.3.2.14 operator>>()

```
istream & operator>> (
            istream & is,
            Studentas & s)
```

Definition at line 604 of file funk.cpp.

5.3.2.15 outputas()

```
void outputas (
            const vector< Studentas > & grupe)
```

Print students to console

Definition at line 120 of file funk.cpp.

5.3.2.16 randominputas()

```
void randominputas (
            vector< Studentas > & grupe,
            bool ArGeneruotiVardus)
```

Definition at line 166 of file funk.cpp.

5.3.2.17 randompavarde()

```
string randompavarde (
            string lytis)
```

Definition at line 324 of file funk.cpp.

5.3.2.18 randomvardas()

```
string randomvardas (
            string lytis)
```

Definition at line 316 of file funk.cpp.

5.3.2.19 rikiuoti()

```
void rikiuoti (
            vector< Studentas > & grupe,
            int pasirinkimas)
```

Definition at line 369 of file funk.cpp.

5.3.2.20 skirstyti()

```
void skirstyti (
            const vector< Studentas > & grupe,
            vector< Studentas > & protingi,
            vector< Studentas > & neprotingi)
```

Definition at line 543 of file funk.cpp.

5.3.2.21 skirstyti2()

```
void skirstyti2 (
            vector< Studentas > & grupe,
            vector< Studentas > & neprotingi)
```

Definition at line 567 of file funk.cpp.

5.3.2.22 skirstyti3()

```
void skirstyti3 (
            vector< Studentas > & grupe,
            vector< Studentas > & neprotingi)
```

Definition at line 577 of file funk.cpp.

5.3.3 Variable Documentation

5.3.3.1 fgalunes

```
const vector<string> fgalunes = {"iene", "aite", "yte"}
```

Definition at line 14 of file studentai.h.

5.3.3.2 fvardai

```
const vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"}
```

Definition at line 11 of file studentai.h.

5.3.3.3 irasymo_failas

```
const string irasymo_failas = "kursiokai.txt"
```

Definition at line 145 of file studentai.h.

5.3.3.4 irasymo_failas_blogas

```
const string irasymo_failas_blogas = "kursiokai_blogi.txt"
```

Definition at line 147 of file studentai.h.

5.3.3.5 irasymo_failas_geras

```
const string irasymo_failas_geras = "kursiokai_geri.txt"
```

Definition at line 146 of file studentai.h.

5.3.3.6 mgalunes

```
const vector<string> mgalunes = {"as", "aitis"}
```

Definition at line 13 of file studentai.h.

5.3.3.7 mvardai

```
const vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas",
"Arnas"}
```

Definition at line 10 of file studentai.h.

5.3.3.8 pavardes

```
const vector<string> pavardes = {"Maceraus k", "Jankaus k", "Kazlaus k", "Svilpaus k", "Drugeliaus k",
"Briedaus k"}
```

Definition at line 12 of file studentai.h.

5.4 studentai.h

Go to the documentation of this file.

```
00001 #ifndef STUDENTAI_H
00002 #define STUDENTAI_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <algorithm>
00007
00008 using namespace std;
00009
00010 const vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"};
00011 const vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"};
00012 const vector<string> pavardes = {"Maceraus k", "Jankaus k", "Kazlaus k", "Svilpaus k", "Drugeliaus k",
      "Briedaus k"};
00013 const vector<string> mgalunes = {"as", "aitis"};
00014 const vector<string> fgalunes = {"iene", "aite", "yte"};
00015
00016 enum class SkirstymoStrategija
00017 {
00018     PIRMAS,
00019     ANTRAS,
00020     TRECIAS
00021 };
00022
00023 class Zmogus {
00024 protected:
00025     string vardas;
00026     string pavarde;
00027
00028 public:
00029     Zmogus(const string& v = "A", const string& p = "BB")
00030     : vardas(v), pavarde(p)
00031     {}
00032     virtual ~Zmogus() {}
00033
00034     virtual void info() const = 0; // kad veiktu abstrakciai
00035
00036     string& refVardas() { return vardas; }
00037     string& refPavarde() { return pavarde; }
00038     string getVardas() const { return vardas; }
00039     string getPavarde() const { return pavarde; }
00040     void setVardas(const string& v) { vardas = v; }
00041     void setPavarde(const string& p) { pavarde = p; }
00042 };
```

```
00043
00044
00045 class Studentas : public Zmogus {
00046 private:
00047     vector<int> paz;
00048     int egz = 0;
00049     double rez = 0.0;
00050     double medrez = 0.0;
00051
00052 public:
00053     Studentas(const string& v = "A",
00054           const string& p = "BB",
00055           const vector<int>& paz_ = {},
00056           int egz_ = 0)
00057     : Zmogus(),          // zmogaus konstruktorius
00058       paz(paz_),
00059       egz(egz_),
00060       rez(0.0),
00061       medrez(0.0)
00062 {
00063     vardas = v;
00064     pavarde = p;
00065
00066     if (!paz.empty())
00067         skaiciuoti();
00068 }
00069
00070 ~Studentas()
00071 {
00072     paz.clear();
00073 }
00074
00075 // Rule of Five
00076
00077 Studentas(const Studentas& other)
00078     : Zmogus(other),
00079       paz(other.paz),
00080       egz(other.egz),
00081       rez(other.rez),
00082       medrez(other.medrez)
00083 {}
00084
00085 Studentas& operator=(const Studentas& other) {
00086     if (this != &other) {
00087         Zmogus::operator=(other);
00088         paz = other.paz;
00089         egz = other.egz;
00090         rez = other.rez;
00091         medrez = other.medrez;
00092     }
00093     return *this;
00094 }
00095
00096 Studentas(Studentas&& other) noexcept
00097     : Zmogus(std::move(other)),
00098       paz(std::move(other.paz)),
00099       egz(other.egz),
00100       rez(other.rez),
00101       medrez(other.medrez)
00102 {
00103     other.egz = 0;
00104     other.rez = 0;
00105     other.medrez = 0;
00106 }
00107
00108 Studentas& operator=(Studentas&& other) noexcept {
00109     if (this != &other) {
00110         Zmogus::operator=(std::move(other));
00111         paz = std::move(other.paz);
00112         egz = other.egz;
00113         rez = other.rez;
00114         medrez = other.medrez;
00115
00116         other.egz = 0;
00117         other.rez = 0;
00118         other.medrez = 0;
00119     }
00120     return *this;
00121 }
00122
00123     void info() const override { // kad veiktu ne abstrakciai?
00124         cout « "Studentas: " « vardas « " " « pavarde « endl;
00125     }
00126
00127     vector<int>& refPaz() { return paz; }
00128     int& refEgz() { return egz; }
00129     double& refRez() { return rez; }
```

```
00130     double& refMedrez() { return medrez; }
00131     void clearPaz() { paz.clear(); }
00132     double getRez() const { return rez; }
00133     double getMedrez() const { return medrez; }
00134     int getEgz() const { return egz; }
00135
00136     vector<int>& getPaz() { return paz; }
00137
00138     void addPaz(int p) { paz.push_back(p); }
00139     void setEgz(int e) { egz = e; }
00140
00141
00142     void skaiciuoti();
00143 };
00144
00145 const string irasymo_failas = "kursiokai.txt";
00146 const string irasymo_failas_geras = "kursiokai_geri.txt";
00147 const string irasymo_failas_blogas = "kursiokai_blogi.txt";
00148
00149 void inputas(vector<Studentas> &grupe);
00150 void failinputas(vector<Studentas> &grupe, string reading);
00151 void randominputas(vector<Studentas> &grupe, bool ArGeneruotiVardus);
00152 void outputas(const vector<Studentas> &grupe);
00153 void failoutputas(const vector<Studentas> &grupe, string writing);
00154 void failgeneravimas();
00155 void genirasymas(vector<Studentas> &grupe, vector<Studentas> &protingi, vector<Studentas> &neprotingi,
00156     string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija);
00157 string lytgen();
00158 string randomvardas(string lytis);
00159 string randompavarde(string lytis);
00160 void rikiuoti(vector<Studentas> &grupe, int pasirinkimas);
00161 bool cmpVardas(const Studentas &a, const Studentas &b);
00162 bool cmpPavarde(const Studentas &a, const Studentas &b);
00163 bool cmpRez(const Studentas &a, const Studentas &b);
00164 bool cmpMedrez(const Studentas &a, const Studentas &b);
00165 bool cmpEgz(const Studentas &a, const Studentas &b);
00166 void skirstyti(const vector<Studentas>& grupe, vector<Studentas>& protingi, vector<Studentas>&
      neprotingi);
00167 void logResults(const string& container, int size,
00168                 double read_t, double sort_t, double split_t);
00169 void skirstyti2(vector<Studentas>& grupe, vector<Studentas>& neprotingi);
00170 void skirstyti3(vector<Studentas>& grupe, vector<Studentas>& neprotingi);
00171
00172 ostream& operator<<(ostream& os, const Studentas& s);
00173 istream& operator>>(istream& is, Studentas& s);
00174
00175
00176 #endif
```

5.5 versija_v2.0.cpp File Reference

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <string>
#include <cmath>
#include <algorithm>
#include <random>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include "studentai.h"
#include <stdexcept>
```

Functions

- int main ()

5.5.1 Function Documentation

5.5.1.1 main()

```
int main ()
```

Definition at line 17 of file versija_v2.0.cpp.

5.6 versija_v2.0.cpp

Go to the documentation of this file.

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 int main()
00018 {
00019     srand(time(NULL));
00020
00021     vector<Studentas> grupe;
00022     vector<Studentas> protingi;
00023     vector<Studentas> neprotingi;
00024
00025     grupe.reserve(10000000);
00026
00027     string genpas;
00028     cout « "Norite generuoti 5 failus? y/n " « endl;
00029     cin » genpas;
00030     if (genpas == "y") failgeneravimas();
00031
00032     cout « "--- Zmogus kurimo testavimas ---" « endl;
00033
00034     // Zmogus z;
00035
00036     cout « "--- Rule of Five dalykai ---" « endl;
00037
00038     Studentas a;
00039     a.setVardas("Jonas");
00040     a.setPavarde("Jonaitis");
00041     a.addPaz(10);
00042     a.addPaz(9);
00043     a.setEgz(8);
00044     a.skaiciuoti();
00045
00046     cout « setw(20) « "Originalus elementas a: " « a « endl;
00047
00048     Studentas b(a);          // copy constructor
00049     cout « setw(20) « "Copy ctor (b konstruojamas su a kopija): " « b « endl;
00050     a.addPaz(1);
00051     a.skaiciuoti();
00052     cout « setw(20) « "a pakeiciamas: " « a « endl;
00053     cout « setw(20) « "b lieka toks pat: " « b « endl;
00054
00055     Studentas c;
00056     c = a;                    // copy assignment
00057     cout « setw(20) « "Copy assign (c jau egzistuoja ir kopijuoja a): " « c « endl;
00058     a.addPaz(1);
00059     c.addPaz(10);
00060     cout « setw(20) « "a ir c su pazymiu pakeitimais:" « endl;
00061     a.skaiciuoti();
00062     c.skaiciuoti();
00063     cout « setw(20) « "pradinis (a): " « a « endl;
00064     cout « setw(20) « "kopija (c): " « c « endl;
00065
```

```
00066     Studentas d(move(a));  // move constructor
00067     cout « setw(20) « "Move ctor (d inicializuojamas move'inant a): " « d « endl;
00068     cout « setw(20) « "objektas a, is kurio paimta info (a turi buti tuscias): " « endl;
00069     cout « a « endl;
00070
00071     Studentas e;
00072     e = move(b);          // move assignment
00073     cout « setw(20) « "Move assign (e egzistuoja, b -> e): " « e « endl;
00074     b.addPaz(1);
00075     b.skaiciuoti();
00076     cout « setw(20) « "objektas b pakeiciamas" « endl;
00077     cout « setw(20) « "objektas b, is kurio paimta info: " « endl;
00078     cout « b « endl;
00079
00080     cout « setw(20) « "elementas e lygus elementui e ir yra move'inamas (e -> e): " « endl;
00081     e = e; // cia tas edge case kur move'ina i ta pati dalyka
00082     e = move(e);
00083     cout « e « endl;
00084
00085     cout « "=== End of Test ===" « endl « endl;
00086
00087
00088     int pasirinkimas = 4;
00089     string isvedimotipas, skaitymo_pasirinkimas, skaitymo_failas;
00090     cout « "Kokios norite ivesties? (1 - ranka arba is failo, 2 - generuoti tik pazymius, " « endl
00091     « " 3 - generuoti studentu vardus, pavardes ir pazymius, 4 - skaityti sugeneruotus failus; 5 arba
      kitas simbolis - baigti darba)" « endl;
00092     cin » pasirinkimas;
00093
00094     int rikiavimo_pasirinkimas;
00095     cout « "Pagal ka rikiuoti?" « endl
00096     « "1 - vardas" « endl
00097     « "2 - pavarde" « endl
00098     « "3 - galutinis (vidurkis)" « endl
00099     « "4 - galutinis (mediana)" « endl
00100     « "5 - egzaminas" « endl;
00101     cin » rikiavimo_pasirinkimas;
00102
00103
00104     auto input_start = chrono::high_resolution_clock::now();
00105
00106     if (pasirinkimas == 1)
00107     {
00108         cout « "Ar skaityti duomenis is failo? (y - is failo / n - ranka)";
00109         cin » skaitymo_pasirinkimas;
00110         if (skaitymo_pasirinkimas == "y")
00111         {
00112             string skaitymo_failas;
00113             cout « "Iveskite failo pavadinima: ";
00114             cin » skaitymo_failas;
00115             failinputas(grupe, skaitymo_failas);
00116         }
00117         else inputas(grupe);
00118     }
00119     else if (pasirinkimas == 2)
00120     {
00121         randominputas(grupe, false);
00122     }
00123     else if (pasirinkimas == 3)
00124     {
00125         randominputas(grupe, true);
00126     }
00127     else if (pasirinkimas == 4)
00128     {
00129         genirasymas(grupe, protingi, neprotingi, irasymo_failas_geras, irasymo_failas_blogas,
      rikiavimo_pasirinkimas, SkirstymoStrategija::TRECIAS);
00130         return 0;
00131     }
00132     else
00133     {
00134         cout « "darbas baigtas" « endl;
00135         return 0;
00136     }
00137
00138     auto input_end = chrono::high_resolution_clock::now();
00139     chrono::duration<double> input_diff = input_end - input_start;
00140     cout « "Duomenu nuskaitymas uztruko: " « input_diff.count() « " s \n";
00141
00142     cout « "Ar rasyti i faila? (y/n)" « endl;
00143     cin » isvedimotipas;
00144
00145     auto start = std::chrono::high_resolution_clock::now(); // Paleisti
00146
00147     rikiuoti(grupe, rikiavimo_pasirinkimas);
00148     //skirstyti(grupe, protingi, neprotingi)
00149     //skirstyti2(grupe, neprotingi);
00150     skirstyti3(grupe, neprotingi);
```

```
00151
00152     if (isvedimotipas == "y")
00153     {
00154         cout « "pradedu rasyt" « endl;
00155         failoutputas(protingi, "kursiokai_geri.txt");
00156         failoutputas(grupe, "kursiokai_geri_grupe.txt");
00157         failoutputas(neprotingi, "kursiokai_blogi.txt");
00158         cout « "Baigiu rasyt" « endl;
00159     }
00160     else
00161     {
00162         outputas(grupe);
00163     }
00164
00165     // is pavyzdzio
00166     auto end = chrono::high_resolution_clock::now(); // Stabdyti
00167     chrono::duration<double> diff = end-start; // Skirtumas (s)
00168     cout « "Programos rikiavimas ir isvedimas uztruko: "« diff.count() « " s\n";
00169
00170     cout « "program finished." « endl;
00171     return 0;
00172 }
```

Index